

C++ 示例：使用 const 修饰结构体指针传参

C++ 示例代码：const 修饰结构体指针传参（函数内禁止修改结构体）

核心要点：

用 `const 结构体类型*` 作为函数形参，表示：

指针指向的结构体内容只读，函数内不允许修改结构体任何成员，只能读取。

完整可运行代码

代码块

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // 定义学生结构体
6  struct Student
7  {
8      string name;
9      int id;
10     double score;
11 };
12
13 // 函数：接收 const 结构体指针
14 // const Student* 保证：函数内部不能修改结构体成员
15 void showStudent(const Student* pStu)
16 {
17     // ✅ 允许：读取结构体成员
18     cout << "姓名： " << pStu->name << endl;
19     cout << "学号： " << pStu->id << endl;
20     cout << "分数： " << pStu->score << endl;
21
22     // ❌ 禁止：试图修改 const 结构体成员，编译器直接报错
23     // pStu->name = "张三";
24     // pStu->score = 99;
25 }
26
27 int main()
28 {
29     // 定义结构体变量
```

```
30     Student s = {"李四", 2026001, 88.5};
31
32     // 传结构体地址给函数
33     showStudent(&s);
34
35     return 0;
36 }
```

运行结果

代码块

```
1  姓名：李四
2  学号：2026001
3  分数：88.5
```

关键讲解

1. 为什么要用 `const Student*`

- 不加 `const`：`Student* pStu`，函数既能读、也能改结构体内容；
- 加 `const`：`const Student* pStu`，强制只读，编译器保护，杜绝误修改。

2. 访问方式不变

结构体指针访问成员依旧用 `->`，不受 `const` 影响。

3. 三种 `const` 指针区别（必记）

代码块

```
1  // 1. const 修饰指向的内容（最常用）
2  // 指针可改，结构体内容 不可改
3  void fun1(const Student* p);
4
5  // 2. const 修饰指针本身
6  // 指针不可改，结构体内容 可改
7  void fun2(Student* const p);
8
9  // 3. 都加 const
10 // 指针不可改，结构体内容 也不可改
```

```
11 void fun3(const Student* const p);
```

总结

只需要记住固定写法：

想让函数只读结构体、不修改 → 形参写 `const 结构体名*` 指针名

安全、规范，是工程里传结构体的标准写法。

（注：文档部分内容可能由 AI 生成）